



BUSINESS TRAVEL & TOURISM TECHNOLOGY

MODULE 1 · FOUNDATION

DAY 1

Environment Setup, Git & Terminal

The working environment and the daily-push habit that every later day depends on.

Learning Objectives

By the end of today, students will be able to:

1 Work in the terminal

Navigate the filesystem and run core commands with confidence instead of relying on a file manager.

2 Use Git locally

Initialise a repository and move work through the staging area into commits with clear messages.

3 Connect to GitHub

Link a local repo to a remote and run the clone / add / commit / push loop end to end.

4 Adopt the daily-push habit

Push every exercise to GitHub from day one, building a professional habit and a public portfolio.

5 Set up the toolchain

Have VS Code, Git, the terminal, and a GitHub account installed and verified on their own machine.

6 Locate the catch-up path

Understand the placement diagnostic and how targeted homework brings everyone to a common baseline.

Today's Agenda

Four instructional hours, the same rhythm used every day in the program.

Concept & Demo

Why the terminal and Git matter.
Live demo of navigation, init, and
a first commit.

Guided Practice

Walk the class through Git's
three areas and the add →
commit → push loop together.

Independent Lab

Students set up their machine
and push their first repo to
GitHub — the 'Hello, Repo'
exercise.

Review & Homework

Recap, troubleshoot common
errors, peer check, and assign
the reading sheet and catch-up
set.

Welcome to CodeOps

A 24-week, 480-hour professional program that takes you from fundamentals to a deployed full-stack application.

9

Modules

120

Days

480

Hours

3

Backend tracks

Where today fits

Day 1 sets up the ground everything else stands on. Before any Python, before OOP, before a single web page — you learn to work like a developer: in the terminal, with version control, pushing to GitHub. Every exercise and project for the next 119 days will live in Git and be pushed to GitHub. Today is where that habit begins.

SECTION ONE

How This Program Works

The teaching model and the catch-up mechanism — the rules of the road before we touch a single command.



INTERNATIONAL BUSINESS TECHNOLOGY

The Teaching Model

The same loop runs every day. It is the program's main strength — and it depends on doing it consistently.



The daily-push habit: everything you write is committed and pushed to GitHub from the very first session. It backs up your work, builds the professional habit early, and grows into the public portfolio employers actually look at. A larger project also grows alongside each module as new concepts are added.

The Placement Diagnostic & Catch-Up Homework

A mixed intake is normal. The program is intensive — and a short diagnostic plus calibrated homework is what keeps the pace survivable for everyone.

1

A short diagnostic, today

A quick placement check on Day 1 shows who arrives with more coding background and who arrives with less. It is not graded against you — it calibrates your support.

2

Homework closes the gap

Every day ends with homework. In the early modules it is deliberately substantial, so anyone with less background can use it to catch up to the rest of the cohort.

3

Extra, targeted sets

Students who need the most support get additional catch-up homework on top of the daily set — heaviest here in the Foundation module, tapering as the cohort converges.

By the end of Module 1, every student has reached the common baseline the rest of the program depends on. The goal is convergence, not competition.

SECTION TWO

The Command Line

Why developers live in the terminal, and the handful of commands that cover almost everything you'll do day to day.

2



© 2020 INSTITUTE OF BUSINESS TECHNOLOGY

What Is the Terminal?

The terminal is a text interface to your computer.

Instead of clicking icons, you type commands and the computer runs them. It looks bare, but it is faster, scriptable, and exact — and it is how nearly all developer tools are driven.

Git, package managers, servers, cloud tools, and deployment all run from here. Getting comfortable now pays off in every module that follows.

GUI point and click — friendly, but slow and hard to automate.

CLI type commands — precise, fast, repeatable, scriptable.

```
bash – student@codeops
```

```
$ pwd
```

```
/home/student/codeops
```

```
$ ls
```

```
day01  README.md
```

```
$ echo "Hello, CodeOps"
```

```
Hello, CodeOps
```

```
$ _
```

Anatomy of a Command

Every command follows the same shape. Once you can read it, you can read almost any command you meet.

```
git commit -m "first commit"
```

git

PROGRAM

The tool you are running.

commit

SUBCOMMAND

What you want it to do.

-m

FLAG / OPTION

Modifies behaviour (here: provide a message).

"first commit"

ARGUMENT

The value the flag or command acts on.

Navigating the Filesystem

pwd Print working directory — where am I right now?

ls List the contents of the current directory.

ls -la List everything, including hidden files, with detail.

cd folder Change into a directory.

cd .. Move up one level to the parent directory.

cd ~ Jump to your home directory.

Absolute vs. relative paths

Absolute

/home/student/codeops/day01

Starts from the root — works from anywhere.

Relative

day01/exercises

Starts from where you are right now.

Tip: Tab completion finishes file and folder names for you. Use it constantly — it prevents typos and saves time.

Creating, Moving & Removing Files

mkdir day01

Make a new directory.

touch notes.md

Create an empty file (or update its timestamp).

cat notes.md

Print a file's contents to the screen.

cp file copy

Copy a file. Add -r to copy a whole folder.

mv old new

Move a file — or rename it, same command.

rm file

Remove a file. There is no recycle bin.

⚠ **rm deserves respect.** On the command line there is no undo and no trash — when you remove something, it is gone immediately. Never run **rm -rf** unless you are certain what it points at, and read the path twice before pressing Enter.

Shortcuts & a Worked Example

Quality-of-life shortcuts

Tab

Auto-complete file and command names.

↑ / ↓

Scroll through previous commands.

clear

Wipe the screen (history stays).

history

List recently-run commands.

man cmd

Open the manual for any command.

Ctrl + C

Stop a command that's stuck.

Set up a project from nothing

```
$ cd ~
```

```
$ mkdir codeops && cd codeops
```

```
$ mkdir day01 && cd day01
```

```
$ touch README.md
```

```
$ echo "# Day 1" > README.md
```

```
$ ls -la
```

```
total 4
```

```
-rw-r--r--  README.md
```

```
$ cat README.md
```

```
# Day 1
```

SECTION THREE

Version Control with Git

What Git is, the problem it solves, and the core commands you'll use to track every change you make.

3



© 2020 IBT. ALL RIGHTS RESERVED.

The Problem Git Solves

Life without version control

- final.js, final_v2.js, final_REAL.js, final_USE_THIS.js...
- You break something and can't get back to what worked.
- Two people edit the same file and overwrite each other.
- No record of what changed, when, or why.

With Git

- One folder, one history — no more _final_v2 files.
- Every saved snapshot is recoverable. Roll back anytime.
- Changes merge cleanly; conflicts are flagged, not silent.
- A complete, dated log of what changed and why.

What Is Git?

Git is a distributed version control system. It records snapshots of your project over time, so you can see its full history, return to any earlier state, and combine work from many people without losing anything.

1

Snapshots, not copies

Each commit captures the state of your whole project at a moment in time — not a separate copy of every file.

2

Distributed

Every clone is a full copy of the history. You can work, commit, and review entirely offline.

3

Fast & local

Almost everything happens on your own machine. Pushing to GitHub is a separate, deliberate step.

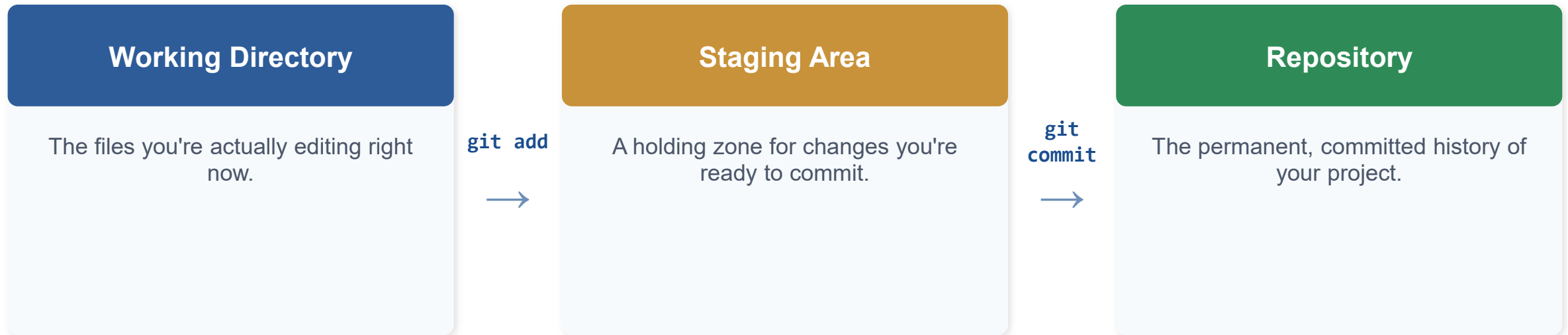
4

Git ≠ GitHub

Git is the tool that runs on your computer. GitHub is a website that hosts Git repositories online.

Git's Three Areas

Understanding these three places is the key to Git. Almost every command moves work from one to the next.



The flow: you edit files (working directory), **stage** the ones you want to save, then **commit** them into the repository as a permanent snapshot.

Starting, Staging & Committing

```
$ git init
```

```
Initialized empty Git repository
```

```
$ git status
```

```
Untracked files: README.md
```

```
$ git add README.md
```

```
$ git commit -m "Add README"
```

```
[main abc1234] Add README
```

```
1 file changed, 1 insertion(+)
```

`git init`

Turn the current folder into a repository.

`git add`

Stage the changes you want to save.

`git commit`

Save a snapshot, with a clear message.

Good messages: say what the change does, in the present tense — **Add login validation**, not **stuff** or **final fix2**.

Seeing History & Ignoring Junk

Seeing what happened

`git log`

Full commit history — who, what, when.

`git log --oneline`

A compact one-line-per-commit view.

`git diff`

What you've changed but not yet staged.

```
$ git log --oneline  
  
abc1234 Add README  
  
9f8e7d6 Initial commit
```

Keeping junk out: .gitignore

Lists patterns Git should skip, so your repo never tracks secrets, dependencies, or build output.

```
.gitignore  
  
# secrets — never commit  
.env  
*.key  
# dependencies & build  
node_modules/  
dist/
```

Why it matters: API keys and .env files must never reach GitHub — a public repo with a live key is a real security incident. If a secret slips in, rotate the key.

SECTION FOUR

GitHub, Workflow & Setup

Connecting to GitHub, the repeatable loop you'll run every session, and the toolchain you set up today.

4



INTERNATIONAL BUSINESS TECHNOLOGY

Local vs. Remote & Authenticating



Local (your computer)

Where you edit, stage, and commit. Fast, private, works offline.

`git push` →

← `git pull`



Remote (GitHub)

A backup and a shareable home for your repo — where your public portfolio lives.

Passwords aren't accepted for Git — authenticate one of two ways:

SSH Keys

RECOMMENDED

A key pair on your machine; add the public half to GitHub once, then push without typing anything again.

Personal Access Token

SIMPLE TO START

A long token that stands in for a password over HTTPS, generated in GitHub settings and pasted when Git asks.

Connecting to GitHub

Create an empty repository on GitHub, then point your local repo at it. 'origin' is just a nickname for that remote URL.

1

Create the repo on GitHub

New → name it → leave it empty (no README).

2

Link it as 'origin'

```
git remote add origin <url>
```

3

Verify the link

```
git remote -v
```

 shows the connected URL.

4

First push

```
git push -u origin main
```

 sets the default.

Clone, Push & Pull

```
git clone <url>
```

Download a remote repo to your machine — your starting point for the program's exercises.

```
git push
```

Send your local commits up to GitHub. This is the daily-push step.

```
git pull
```

Bring down changes from GitHub before you start work.

```
$ git clone https://github.com/student/codeops.git
Cloning into 'codeops'... done.
$ git push
```

Your Daily Git Workflow

This is the loop you will run at the end of every single session for the rest of the program. Memorise it.



Pull, work, add, commit, push. Five steps, every day. By Module 2 it will be muscle memory — exactly the habit employers expect.

Your Day-1 Toolchain

Install and verify these today. Each one comes back again and again across the program.

VS Code

YOUR CODE EDITOR

Where you'll write everything.
Add extensions as you go.

```
code --version
```

Git

VERSION CONTROL

The tool behind every commit
and push in the program.

```
git --version
```

Terminal

COMMAND LINE

Built in on Mac/Linux; use Git
Bash or WSL on Windows.

```
echo $SHELL
```

GitHub account

YOUR PORTFOLIO HOME

Free account. This is where your
public work will live.

```
(sign up online)
```

Verify each install by running the command on its card — if it prints a version, you're set.

Day 1 in Review

1

The teaching model

Teach → read → build → push. The same loop runs every day of the program.

2

The terminal

A precise, scriptable text interface — pwd, ls, cd, mkdir, touch, and friends.

3

Git's three areas

Working directory → staging → repository, moved by add and commit.

4

Core Git commands

init, status, add, commit, log, diff — the everyday toolkit.

5

GitHub workflow

Connect a remote, then pull → add → commit → push, every session.

6

Your toolchain

VS Code, Git, terminal, and a GitHub account — installed and verified.

Review Questions

Q1

What is the difference between Git and GitHub?

Q2

Name Git's three areas and the command that moves work between each.

Q3

What does git add do, and how is it different from git commit?

Q4

Why is git pull the first step in the daily workflow?

Q5

What belongs in a .gitignore file, and why keep secrets out of a repo?

Q6

Walk through the five steps of the daily Git workflow in order.

In-Class Exercise — "Hello, Repo"

Goal

Set up a fresh project, put it under version control, and push your first commit to GitHub — the whole loop, end to end.

Steps

1. Create a folder hello-repo and a README.md inside it.
2. Run git init and confirm with git status.
3. Stage and commit the README with a clear message.
4. Create an empty repo on GitHub and add it as origin.
5. Push, then refresh GitHub to see your file online.

Deliverable

A GitHub repo URL with at least one commit, pushed from your machine.

Provided files

- Starter/ — skeleton + TODOs
- Solution/ — worked reference
- Rubric.docx — how it's graded
- commands-cheatsheet.md

Homework & Reading

The reading assignment is the assigned work and lives on its own Day 1 reading sheet. The Learning Resources next slide are optional, supplementary depth.

1 Reading Assignment

Git & terminal revision — the Day 1 reading sheet. Read it, then complete the short exercise it ships with.

2 Daily Practice

Recreate today's terminal walkthrough from memory and push it to your GitHub as day01.

3 Catch-Up Set

Targeted extra practice for students flagged by the placement diagnostic — basics drilled before Day 2.

4 Prepare for Day 2

Confirm Python is installed (`python --version`). Day 2 is Python fundamentals.

Learning Resources

Optional, supplementary depth — not the assigned reading. Reach for these when you want more than today covered.

A Pro Git (book)

Free, official, and thorough. The reference for everything Git.

git-scm.com/book

B GitHub Docs

Clear guides for SSH keys, tokens, and your first repository.

docs.github.com

C The Missing Semester

MIT's practical course on the terminal and developer tools.

missing.csail.mit.edu

D Oh My Git!

A free game that teaches Git visually and interactively.

ohmygit.org

E Learn Git Branching

Visual, in-browser practice of Git commands.

learngitbranching.js.org

F explainshell

Paste any command to see what each part does.

explainshell.com